

# “小米杯” 初赛技术设计文档



学校名称： 武汉大学

作品编号： T2026104869910181

队伍名称： 能做到

队伍成员： 杨文韬 李国峰

指导老师： 黄建忠

完成日期： 2026年5月31日

# 目录

|                     |           |
|---------------------|-----------|
| <b>1 赛题分析</b>       | <b>4</b>  |
| 1.1 比赛背景            | 4         |
| 1.2 赛道组成与任务要求       | 4         |
| 1.3 六个赛段任务分析        | 6         |
| 1.4 系统设计难点          | 7         |
| <b>2 系统总体设计</b>     | <b>7</b>  |
| 2.1 总体设计思路          | 7         |
| 2.2 系统架构            | 8         |
| 2.3 软件模块划分          | 9         |
| 2.4 系统运行流程          | 9         |
| <b>3 关键技术设计</b>     | <b>10</b> |
| 3.1 多传感器感知与定位技术     | 10        |
| 3.1.1 ROS2 感知节点设计   | 10        |
| 3.1.2 传感器数据接入       | 11        |
| 3.1.3 坐标校准与定位使用     | 11        |
| 3.1.4 目标检测与降级策略     | 11        |
| 3.2 世界坐标导航与路径规划技术   | 12        |
| 3.2.1 赛道坐标建模        | 12        |
| 3.2.2 世界坐标误差到机体坐标误差 | 12        |
| 3.2.3 waypoint 跟踪控制 | 12        |
| 3.2.4 LiDAR 横向居中辅助  | 13        |
| 3.2.5 卡滞检测与恢复       | 13        |
| 3.3 任务状态机调度技术       | 13        |
| 3.3.1 状态定义          | 13        |
| 3.3.2 赛段调度方式        | 14        |
| 3.3.3 超时与错误处理       | 14        |
| 3.4 LCM 运动控制技术      | 14        |
| 3.4.1 LCM 通信封装      | 14        |
| 3.4.2 速度、姿态与步态控制    | 15        |
| 3.5 特殊步态与姿态控制技术     | 16        |
| 3.5.1 高抬腿通过石板       | 16        |
| 3.5.2 低姿态通过限高杆      | 16        |
| 3.5.3 倾斜桥面横滚补偿      | 16        |
| 3.5.4 宽窄步态控球        | 17        |

|                          |           |
|--------------------------|-----------|
| 目录                       | 3         |
| <b>4 分赛段策略设计</b>         | <b>17</b> |
| 4.1 第一赛段：石径探路 . . . . .  | 17        |
| 4.2 第二赛段：荒野寻珠 . . . . .  | 17        |
| 4.3 第三赛段：曲道冲锋 . . . . .  | 18        |
| 4.4 第四赛段：深隧寻珍 . . . . .  | 19        |
| 4.5 第五赛段：孤梁稳渡 . . . . .  | 20        |
| 4.6 第六赛段：撷金建功 . . . . .  | 21        |
| <b>5 系统实现</b>            | <b>22</b> |
| 5.1 项目代码结构 . . . . .     | 22        |
| 5.2 主程序入口实现 . . . . .    | 23        |
| 5.3 状态机实现 . . . . .      | 23        |
| 5.4 分赛段模块实现 . . . . .    | 23        |
| 5.5 配置文件与参数管理 . . . . .  | 24        |
| 5.6 语音播报实现 . . . . .     | 24        |
| <b>6 实验结果与分析</b>         | <b>24</b> |
| 6.1 仿真测试结果 . . . . .     | 24        |
| 6.2 各赛段通过情况 . . . . .    | 25        |
| 6.3 全流程耗时分析 . . . . .    | 26        |
| 6.4 稳定性分析 . . . . .      | 26        |
| 6.5 问题与改进 . . . . .      | 27        |
| <b>7 总结与展望</b>           | <b>27</b> |
| <b>A 主要参数表</b>           | <b>27</b> |
| <b>B 代码目录结构</b>          | <b>28</b> |
| <b>C 关键函数说明</b>          | <b>29</b> |
| <b>D 赛段 waypoint 配置表</b> | <b>29</b> |

# 1 赛题分析

## 1.1 比赛背景

本届比赛聚焦四足机器人控制和具身传感器使用，要求参赛队伍围绕真实赛道设计一套能够自主运行的智能系统。赛题名称为“荒野寻宝”，任务包含探路、寻物、交互、复杂地形通过和终点动作。与单一的导航或识别任务相比，本题更接近一个小型的综合机器人系统：机器人不仅要知道自己在哪里、下一步去哪里，还要根据不同地形选择不同步态，根据目标物体采取不同动作，并在整个过程中保持稳定。

从工程角度看，这类任务的现实意义在于，它不是只考察某一个算法指标，而是考察系统是否能在约束条件下完成连续任务。四足机器人未来进入巡检、搜救、物流、陪伴和教育等场景时，也会遇到类似问题：环境并不总是平整，目标并不总是居中，传感器并不总是稳定，控制命令也不可能完全按理想模型执行。因此，本项目的设计目标可以概括为：在可解释、可调试的框架下，把感知、导航、动作和任务管理组织成一个能够持续工作的整体。

## 1.2 赛道组成与任务要求

赛道整体由六个赛段组成，机器人从起点趴下状态开始，站起后计时。比赛过程中不允许人工再次操作电脑或机器人，所有任务必须由程序自主完成。六个赛段分别为：

1. 石径探路：从起点出发，通过石板路并进入下一赛段。
2. 荒野寻珠：在小球阵列中寻找指定颜色的橙色小球，并通过身体撞击使其晃动。
3. 曲道冲锋：沿曲线、直线、曲线组合赛道行进，不踩线越线。
4. 深隧寻珍：在通道中识别目标物体和障碍物，完成语音播报与交互动作。
5. 孤梁稳渡：在独木桥和倾斜桥面上行走，到达终点前跳下。
6. 撷金建功：将足球从出口位置带出或踢出，进入终点区域并趴下。



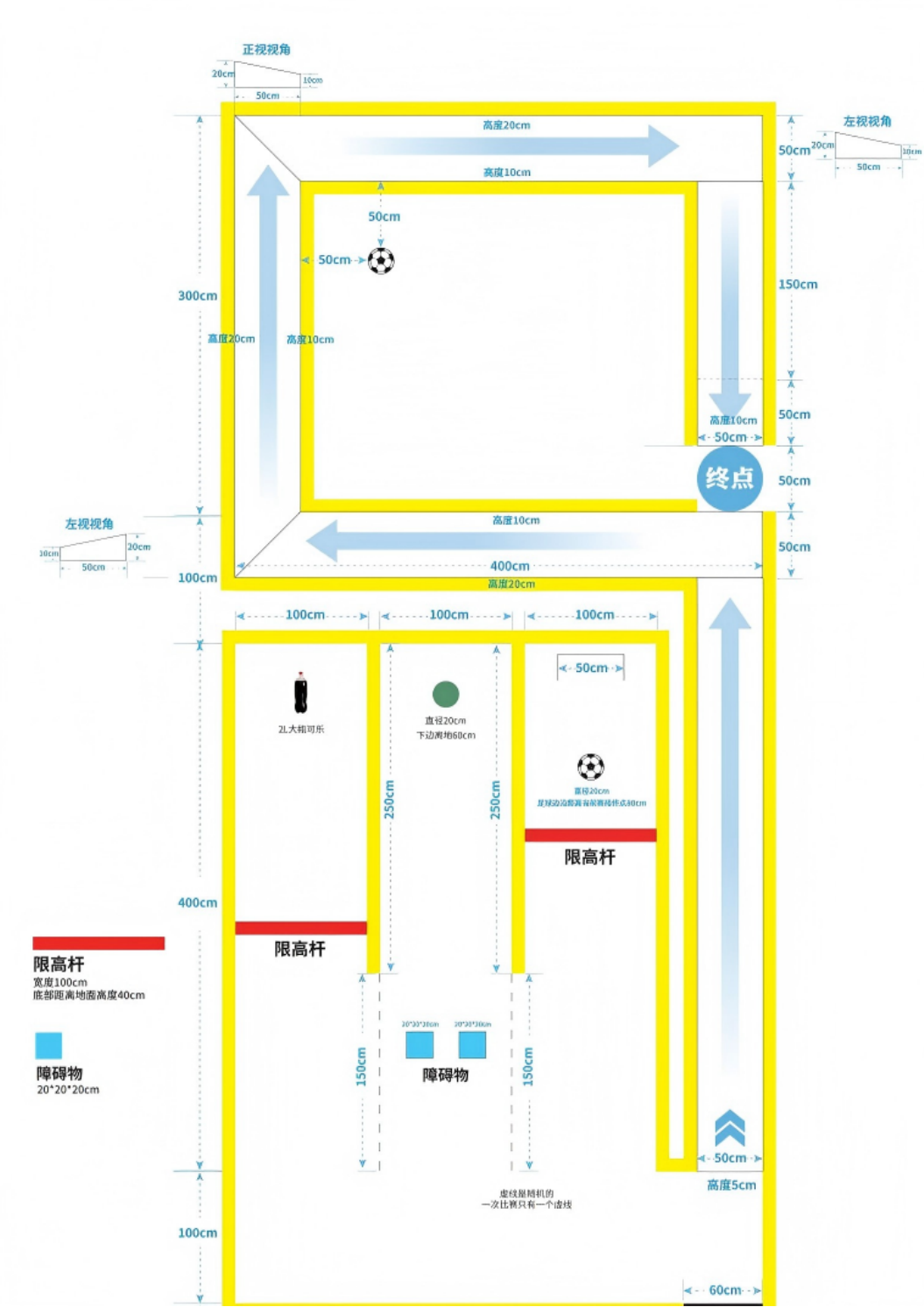


图 1: 比赛赛道整体布局图 1

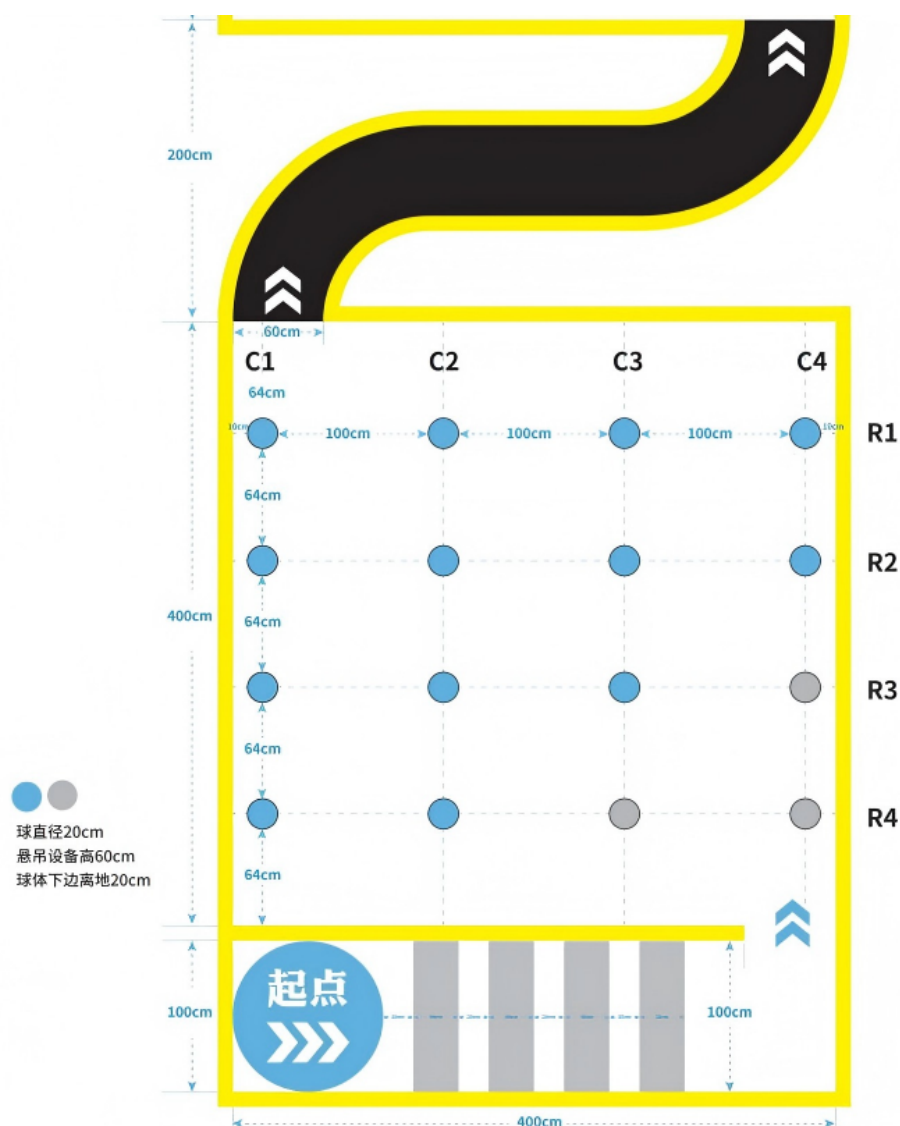


图 2: 比赛赛道整体布局图 2

### 1.3 六个赛段任务分析

第一赛段的核心是稳定通过石板。石板有高度和间隔，普通平地步态容易发生碰撞或卡脚，因此需要提高步高，同时保持方向稳定。

第二赛段的核心是目标选择和撞击。赛题要求找到橙色小球，而普通小球为浅蓝色。理想情况下应使用视觉识别区分颜色；在仿真图像不可用或识别不稳定时，仍需要有可执行的保底路径。

第三赛段的难点在于弯道连续行进。该段不需要复杂交互，但要求机器人沿弯道稳定前进，不能因为过度转向或定位误差导致越界。

第四赛段综合了识别、避障、语音和动作。机器人需要在狭窄通道中寻找可乐瓶、橙色小球和足球，同时识别限高杆与无法跨越的障碍。该段是感知与动作结合最紧密

的赛段。

第五赛段要求机器人通过独木桥和倾斜桥面。该段不仅要求位置精度，更要求姿态稳定。机器人在侧倾面上行进时，横滚角会影响足端支撑和重心位置，因此需要通过 IMU 反馈进行补偿。

第六赛段要求处理足球并到达终点。足球与机器人腿部、身体之间存在接触不确定性，单纯“踢一脚”的方式很难稳定完成。项目中采用宽步态接近、窄步态带球、出口释放的策略，使足球处理过程更可控。

## 1.4 系统设计难点

本项目面对的困难主要体现在以下几个方面。

首先，比赛是连续任务，不是单点测试。某一赛段结束时的位姿误差会影响下一赛段，如果每段都假定机器人处于理想起点，系统很容易在中途失效。因此，整体设计必须允许一定的起点误差，并在每段开始时进行必要的重定位或姿态调整。

其次，感知条件并不总是完整。RGB 图像、深度图、雷达、里程计等数据在仿真和实机中可能存在延迟、缺失或噪声。系统不能把所有决策都压在单一传感器上，而应设计最小可运行条件和降级策略。

再次，四足机器人的运动控制有明显的物理约束。转向过快、步高过低、身体姿态不合适，都可能导致碰撞、滑移或摔倒。因此，路径规划和动作设计不能脱离步态能力。

最后，比赛时间有限。系统既要稳定，又不能过慢。各赛段策略需要在“完成任务”和“节省时间”之间取得平衡，避免为了追求单段精度而牺牲整体流程。

# 2 系统总体设计

## 2.1 总体设计思路

本项目采用分层、分段、闭环的设计思路。分层是指把系统拆成感知、决策、导航、运动控制和赛段执行几个相对清晰的模块；分段是指按照赛题的六个赛段分别设计策略；闭环是指每个动作都尽量依赖当前位置、航向或姿态反馈，而不是只按固定时间盲走。

这样的设计不是为了形式上的整齐，而是为了便于调试和迭代。四足机器人比赛中，许多问题不是一次性通过理论推导解决的，而是在反复测试中逐渐暴露。例如，某段路点看起来合理，但机器人实际到达时会有横向偏差；某个步态在平地上稳定，但在斜坡上容易滑移。分层和分段可以让这些问题被定位在较小范围内，便于修改参数和替换策略。

## 2.2 系统架构

系统总体架构如图所示。主程序 `race_main.py` 负责启动 LCM 控制器、ROS2 感知节点和语音模块，并创建任务状态机。状态机按照比赛顺序调用六个赛段的 `execute()` 函数。每个赛段根据自身任务选择导航辅助函数、感知检测器和运动控制命令。

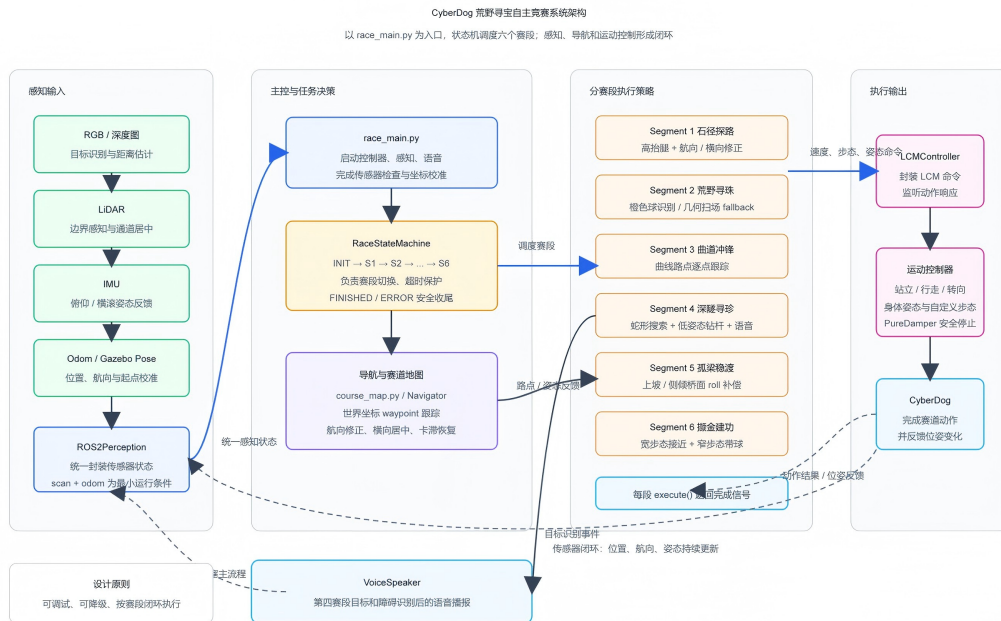


图 3: 系统总体架构图

系统分为以下几层：

- 感知层：通过 `ROS2Perception` 统一接入 RGB 图像、深度图、激光雷达、IMU、里程计和 Gazebo 位姿。
- 决策层：通过 `RaceStateMachine` 管理比赛状态，负责赛段切换、超时控制和错误处理。
- 导航层：通过 `course_map.py`、`Navigator` 和 `nav_helper.py` 实现世界坐标路点导航。
- 运动控制层：通过 `LCMController` 封装 LCM 通信，向底层运动控制器发送步态、速度和姿态命令。
- 赛段执行层：通过 `segments/` 目录下的六个赛段文件实现具体过关策略。

2.3 软件模块划分

表 1: 核心软件模块说明

| 模块                 | 作用                                                          |
|--------------------|-------------------------------------------------------------|
| race_main.py       | 程序入口，负责参数解析、控制器启动、感知线程启动、位置校准和状态机运行。                        |
| state_machine.py   | 定义比赛状态，按 INIT、SEGMENT_1 至 SEGMENT_6、FINISHED、ERROR 的顺序调度任务。 |
| ros2_perception.py | 封装传感器数据订阅，提供位置、航向、图像、深度、雷达和姿态信息。                            |
| course_map.py      | 定义赛道世界坐标、赛段入口出口和主要路点。                                       |
| navigator.py       | 提供通用路点跟踪、偏离检测和卡滞恢复框架。                                       |
| nav_helper.py      | 提供更直接的导航原语，如到点、转向、斜坡转向和雷达横向修正。                              |
| lcm_controller.py  | 封装 LCM 命令发送、响应监听、站立、趴下、步态和用户自定义动作。                          |
| segments/          | 每个文件对应一个赛段，包含该赛段的路径、动作、参数和完成条件。                             |

2.4 系统运行流程

系统启动后，主程序首先创建 LCM 控制器并启动发送、接收线程。随后根据运行模式创建感知节点。在仿真模式下，系统等待传感器就绪；其中激光雷达和里程计是最小运行条件，RGB 和深度图如果未就绪，系统会给出警告，并允许部分视觉任务采用降级策略继续运行。

传感器就绪后，系统读取 Gazebo 中机器人初始位置，并将其映射到赛道坐标系。这样即使仿真世界中的机器人出生点不是 (0, 0)，导航模块仍可以按照统一的 `course_map` 坐标工作。

随后，状态机进入 INIT 状态，机器人站立并进入第一赛段。每个赛段执行成功后返回 `True`，状态机再切换到下一赛段。若比赛超出总时间限制，状态机进入结束状态；若执行过程中出现异常，则进入 ERROR 状态并让机器人进入安全阻尼状态。

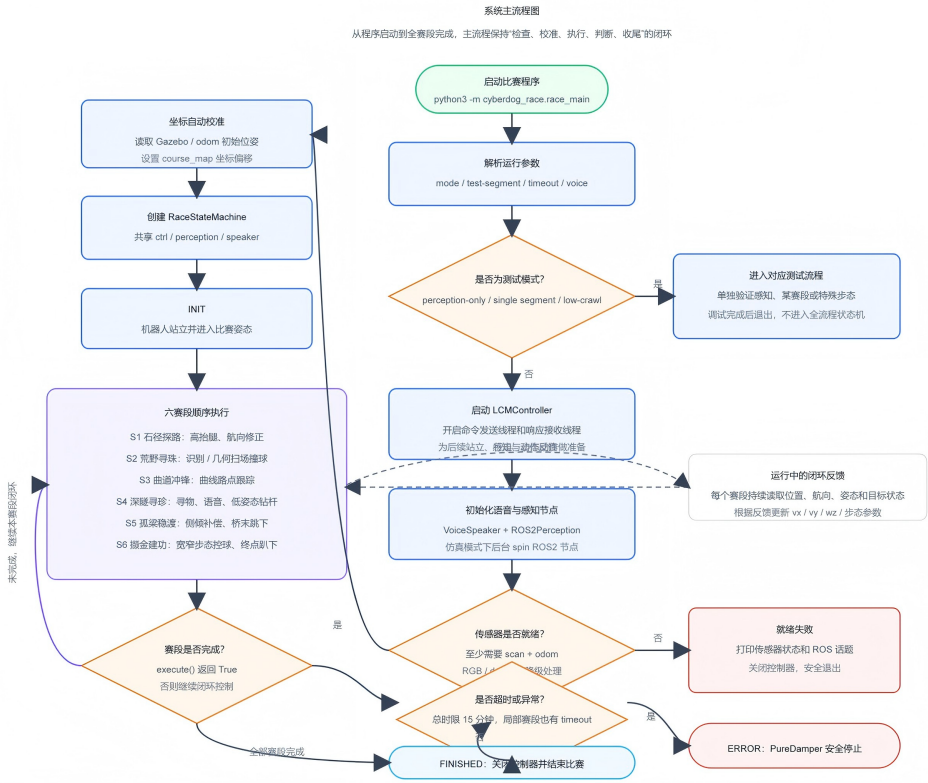


图 4: 系统主流程图

### 3 关键技术设计

#### 3.1 多传感器感知与定位技术

##### 3.1.1 ROS2 感知节点设计

项目使用 ROS2Perception 作为统一感知入口。该节点订阅相机、深度、激光雷达、IMU、里程计和 Gazebo 位姿相关话题，并将它们整理为上层可以直接调用的属性。例如，赛段策略不需要关心图像话题名称，也不需要重复处理四元数转欧拉角，只需要读取 perception.position、perception.heading、perception.latest\_rgb 等信息。

这种封装降低了赛段策略的复杂度。每个赛段面对的是“当前位置、当前航向、当前图像、当前姿态”这些任务层概念，而不是分散的 ROS2 消息。这使得代码更容易阅读，也方便在仿真和实机之间切换。

3.1.2 传感器数据接入

表 2: 主要传感器及用途

| 数据来源      | 对应信息      | 主要用途               |
|-----------|-----------|--------------------|
| RGB 图像    | 颜色与外观     | 检测橙色小球、足球、可乐瓶和障碍物。 |
| 深度图       | 像素距离      | 辅助估计目标距离，判断接近程度。   |
| 激光雷达      | 周围距离      | 通道居中、边界辅助和障碍感知。    |
| IMU       | 横滚、俯仰、角速度 | 倾斜桥面姿态补偿，判断身体姿态。   |
| 里程计       | 平面位置与航向   | 常规导航反馈。            |
| Gazebo 位姿 | 仿真绝对位姿    | 仿真中提供稳定的全局位置校准。    |

3.1.3 坐标校准与定位使用

项目采用世界坐标作为全局导航基础。赛道地图中约定  $X$  轴向右， $Y$  轴向前，航向角以角度表示，其中  $0^\circ$  表示朝  $+X$ ， $90^\circ$  表示朝  $+Y$ ， $180^\circ$  表示朝  $-X$ 。

在仿真中，机器人实际生成位置可能与赛道地图原点不同。主程序启动时会读取 Gazebo 原始位姿，并计算位置偏移量：

$$\Delta x = -x_{\text{raw}}, \quad \Delta y = -y_{\text{raw}}$$

之后感知层对外提供的位置为：

$$x = x_{\text{raw}} + \Delta x, \quad y = y_{\text{raw}} + \Delta y$$

这样，赛段策略始终可以按照赛道地图的坐标编写，而不必在每个文件里分别处理仿真出生点差异。这个处理看似简单，但对连续赛段很重要。没有统一坐标，后续路点会随着初始偏差整体漂移，调参也会失去一致基准。

3.1.4 目标检测与降级策略

项目中保留了小球检测、物体检测、边界检测和独木桥检测等模块。对于目标识别任务，理想策略是通过 RGB 图像提取颜色、形状和目标区域，再结合深度或估算模型判断距离。

但实际比赛和仿真中，视觉数据可能受光照、话题延迟或渲染状态影响。为避免系统因单个传感器不可用而完全停止，项目采用了降级策略。例如第二赛段在 RGB 图像不可用时，会转为基于赛道几何的扫场路径，按照已知小球阵列位置依次靠近和撞击

目标点。该策略不能替代真正的视觉识别，但能保证系统在感知不完整时仍具有基本行动能力。

这种设计体现了本项目的取舍：在比赛系统中，鲁棒性有时比单点算法的精巧更重要。只要降级策略的边界清楚、行为可预期，就能为全流程通过提供额外保障。

## 3.2 世界坐标导航与路径规划技术

### 3.2.1 赛道坐标建模

项目在 `course_map.py` 中定义了赛段入口、出口和主要路点。各赛段的策略可以直接引用这些坐标，也可以根据具体地形在本赛段文件中定义更细的控制点。

例如，第一赛段从  $(0,0)$  出发，沿  $+X$  方向通过石板，到达第二赛段入口附近后转向  $+Y$ 。第三赛段使用一组沿曲道采样的 waypoint，使机器人逐步贴合曲线路径。第五赛段则在赛段文件中定义了上坡、侧倾和平地五段路径，因为该段不仅要到点，还要根据地面形态切换步态和姿态控制。

### 3.2.2 世界坐标误差到机体坐标误差

导航控制的核心是把目标点与当前位置的世界坐标误差转换到机器人机体坐标系。设目标点为  $(x_t, y_t)$ ，机器人当前位置为  $(x, y)$ ，航向角为  $\theta$ ，则世界坐标误差为：

$$\Delta x = x_t - x, \quad \Delta y = y_t - y$$

将其旋转到机体坐标系后得到：

$$e_x = \Delta x \cos(-\theta) - \Delta y \sin(-\theta)$$

$$e_y = \Delta x \sin(-\theta) + \Delta y \cos(-\theta)$$

其中  $e_x$  表示目标在机器人前后方向上的误差， $e_y$  表示左右方向上的误差。控制器再根据  $e_x$ 、 $e_y$  和航向误差生成前向速度  $v_x$ 、横向速度  $v_y$  和角速度  $\omega_z$ 。

这种方法的优点是直观、稳定、便于调参。它没有依赖复杂的全局规划器，而是利用赛道结构已知这一条件，将规划问题转化为一系列局部可控的到点问题。

### 3.2.3 waypoint 跟踪控制

每个 waypoint 都有位置容差和必要时的目标航向。当机器人距离目标点小于容差时，系统认为该点到达，再进入下一个 waypoint 或执行转向动作。速度根据距离进行限制，距离较远时保持较高速度，接近目标时降低速度，避免因惯性或控制延迟越过目标。



航向角误差采用归一化处理，保证误差落在  $[-180^\circ, 180^\circ]$  内。这样机器人总是选择较短方向转向，不会出现绕远旋转。

#### 3.2.4 LiDAR 横向居中辅助

在石板、通道等狭窄区域，仅靠里程计可能会积累横向偏差。项目在导航辅助函数中加入了基于激光雷达的横向修正：读取左右侧一定角度范围内的距离，估计机器人相对通道中心的偏差，再给横向速度  $v_y$  添加一个小的修正量。

该策略不试图重建完整环境，只解决一个具体问题：机器人在通道或边界之间行进时，需要尽量保持居中。它的参数较小，主要起辅助作用，避免对主导航造成过度干扰。

#### 3.2.5 卡滞检测与恢复

通用导航器中包含卡滞检测逻辑。当机器人长时间位置变化很小，且仍未到达目标点时，系统认为可能发生卡滞。恢复策略通常包括短暂后退、调整方向、重新朝目标点前进。虽然该策略不能解决所有物理碰撞问题，但可以处理一部分轻微卡住或方向不佳导致的停滞。

### 3.3 任务状态机调度技术

#### 3.3.1 状态定义

比赛流程通过 `RaceStateMachine` 管理。状态包括：

- IDLE：空闲状态。
- INIT：机器人站起并准备比赛。
- SEGMENT\_1 至 SEGMENT\_6：六个赛段执行状态。
- FINISHED：比赛结束状态。
- ERROR：异常处理状态。

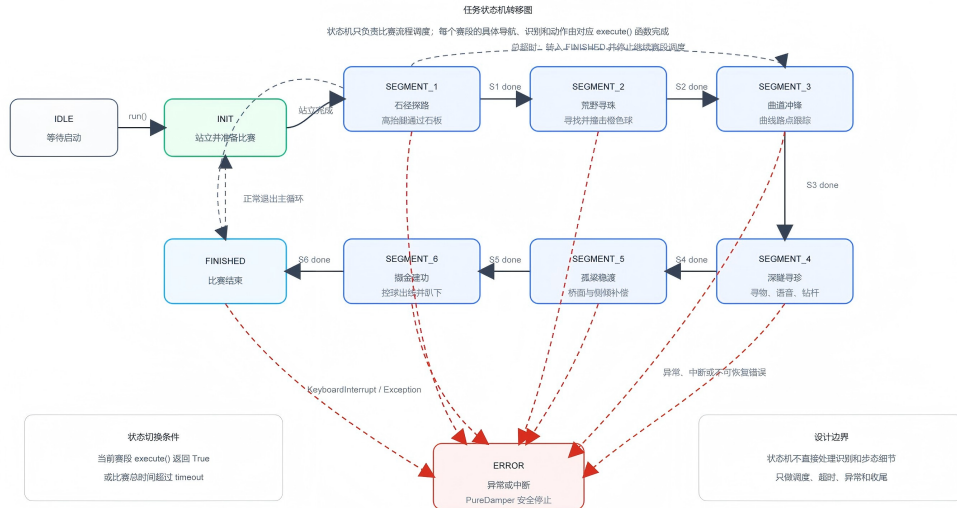


图 5: 任务状态机转移图

### 3.3.2 赛段调度方式

状态机的每个赛段处理函数只做一件事：导入对应赛段的 `execute()` 函数并调用。如果该函数返回完成信号，状态机切换到下一赛段。例如第一赛段完成后进入第二赛段，第六赛段完成后进入 `FINISHED`。

这种接口设计让各赛段具有相对独立性。赛段内部可以根据自身需要使用视觉、路点、特殊步态或固定动作，而状态机不需要了解细节。与此同时，所有赛段仍共享同一套控制器、感知对象、语音对象和导航对象，保证系统上下文连续。

### 3.3.3 超时与错误处理

状态机设置总比赛超时，默认对应赛题的 15 分钟限制。每个赛段内部也设置本段超时时间，防止机器人在某个局部任务中无限等待。若发生异常，状态机进入 `ERROR` 状态，并调用控制器进入阻尼模式，使机器人尽快停止。

超时并不代表设计失败，而是比赛系统必须具备的边界条件。机器人运行环境存在不确定性，如果没有明确的退出条件，单个未完成动作可能拖垮整个比赛流程。

## 3.4 LCM 运动控制技术

### 3.4.1 LCM 通信封装

底层运动控制通过 LCM 消息完成。项目使用 `LCMController` 封装命令发送和响应接收。控制器维护发送线程和接收线程：发送线程按固定频率发布当前命令，接收线程监听机器人控制响应，并记录当前模式、步态和动作进度。

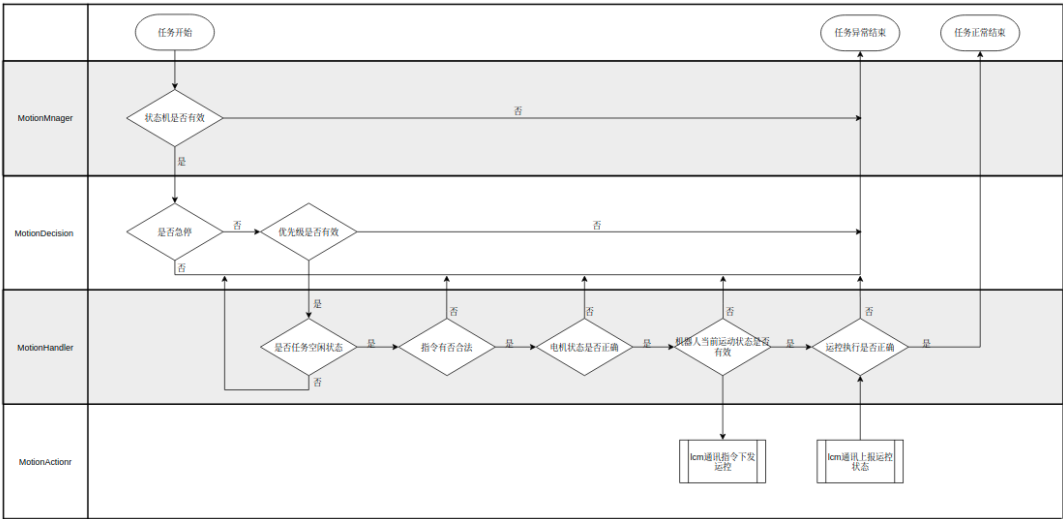


图 6: LCM 控制链路示意图

对上层而言，LCMController 提供的是更接近任务动作的接口，例如：

- stand\_up\_and\_wait(): 站立并等待稳定。
- pure\_damper(): 进入阻尼或趴下状态。
- locomotion(): 发送速度和步态命令。
- set\_height(): 调整身体高度。
- set\_body\_pose(): 同时调整身体高度和俯仰角。
- execute\_gait\_steps(): 执行用户自定义步态序列。

3.4.2 速度、姿态与步态控制

四足机器人运动控制中，速度并不是唯一变量。通过石板时需要更高步高，通过限高杆时需要降低身体高度，通过倾斜桥面时需要横滚补偿，通过足球时还要调整左右腿间距。因此，项目在各赛段中根据地形和任务选择不同步态参数。

表 3: 关键步态及应用场景

| 步态或控制方式 | 主要应用     | 设计目的                 |
|---------|----------|----------------------|
| 慢速小跑    | 常规路点导航   | 保持稳定，便于修正方向。         |
| 高抬腿步态   | 第一赛段石板   | 增大足端离地高度，降低碰撞风险。     |
| 低姿态步态   | 第四赛段限高杆  | 降低身体高度和头部高度，从杆下通过。   |
| 横滚补偿步态  | 第五赛段倾斜桥面 | 抵消侧倾影响，维持重心稳定。       |
| 宽步态     | 第六赛段接近足球 | 让机器人跨过或包围足球，避免直接踢偏。  |
| 窄步态     | 第六赛段带球   | 缩小腿间空间，使足球随机器人向出口移动。 |

3.5 特殊步态与姿态控制技术

3.5.1 高抬腿通过石板

第一赛段石板高度约 5 cm，间隔较短。普通步态在足端摆动高度不足时容易碰到石板边缘。项目在该段提高步高，并降低速度，使机器人以更保守的方式通过。与此同时，系统持续根据航向误差修正角速度，根据 LiDAR 侧向信息进行轻微横移，避免走偏。

3.5.2 低姿态通过限高杆

第四赛段的限高杆底部距地面较低，机器人必须降低身体高度并调整俯仰角。项目设计了低姿态步态，核心思想是降低身体质心，同时使用较小步高、较慢速度和连续的自定义步态序列通过限高区域。该段控制不追求速度，而优先保证不碰杆和不失稳。

3.5.3 倾斜桥面横滚补偿

第五赛段侧倾桥面对四足机器人很不友好。若机器人仍按平地姿态行走，身体会向低侧倾斜，足端支撑和重心位置都会变差。项目读取 IMU 或位姿中的横滚角，并按比例生成反向补偿：

$$\phi_{\text{cmd}} = \text{clip}(-k\phi_{\text{meas}}, \phi_{\text{min}}, \phi_{\text{max}})$$

其中  $\phi_{\text{meas}}$  为测得横滚角， $k$  为补偿增益，clip 用于限制补偿角范围。实际实现中，补偿通过用户步态或身体姿态命令施加，并与较小的前向速度和横向修正配合使用。

### 3.5.4 宽窄步态控球

第六赛段没有采用单次大力踢球的思路，而是把足球处理拆成三个阶段。第一阶段使用宽步态接近并穿过足球位置，使机器人获得合适的相对位置；第二阶段切换到窄步态，把足球限制在腿部和身体附近，带向出口；第三阶段再向前释放足球并移动到终点趴下。

这种做法更符合实际控制条件。足球与机器人接触时存在随机性，单次踢球容易因角度偏差失败，而“带球到出口”可以通过连续反馈修正路线。

## 4 分赛段策略设计

### 4.1 第一赛段：石径探路

第一赛段从起点出发，机器人需要通过石板路并转向进入第二赛段。项目将该段拆成三个阶段：首先沿  $+X$  方向通过石板区域；然后继续前进到第二赛段入口附近；最后原地转向到  $90^\circ$ ，为进入第二赛段做准备。

通过石板时，控制策略重点是“慢、稳、高步高”。机器人使用慢速小跑，设置较大的步高，并根据里程计航向进行比例修正。若机器人向左右偏移，LiDAR 横向修正会给出小幅横移速度，帮助机器人保持在可通行区域内。

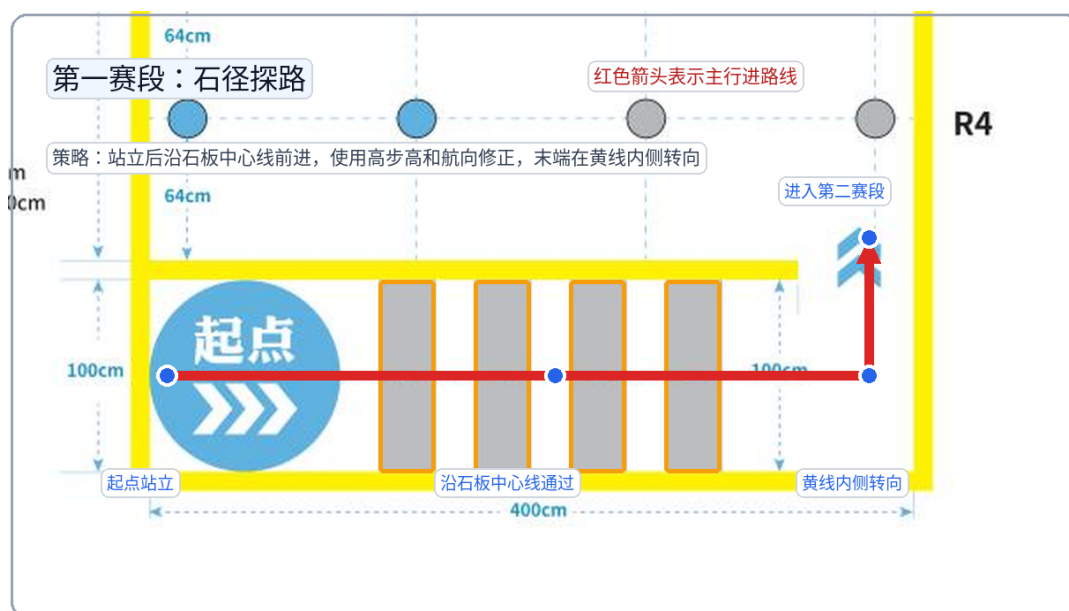


图 7: 第一赛段石板通过路径

### 4.2 第二赛段：荒野寻珠

第二赛段要求在小球阵列中寻找橙色球并撞击。项目保留了视觉搜索模式，也实现了几何扫场模式。在视觉可靠时，系统通过小球检测器识别橙色目标，选择最近且

未处理的目标接近，并在距离合适时执行撞击动作。撞击后，系统记录目标状态，避免重复处理同一小球。

考虑到仿真中 RGB 图像可能不可用，当前策略默认采用几何扫场模式。该模式根据赛道中已知的小球位置，依次前往预设接近点，调整航向后执行短距离撞击。虽然这种策略依赖赛道几何先验，但在比赛规则给出小球阵列结构的情况下，它是合理的保底方案。

完成四个目标点处理后，机器人沿出口 waypoint 前往第三赛段入口。

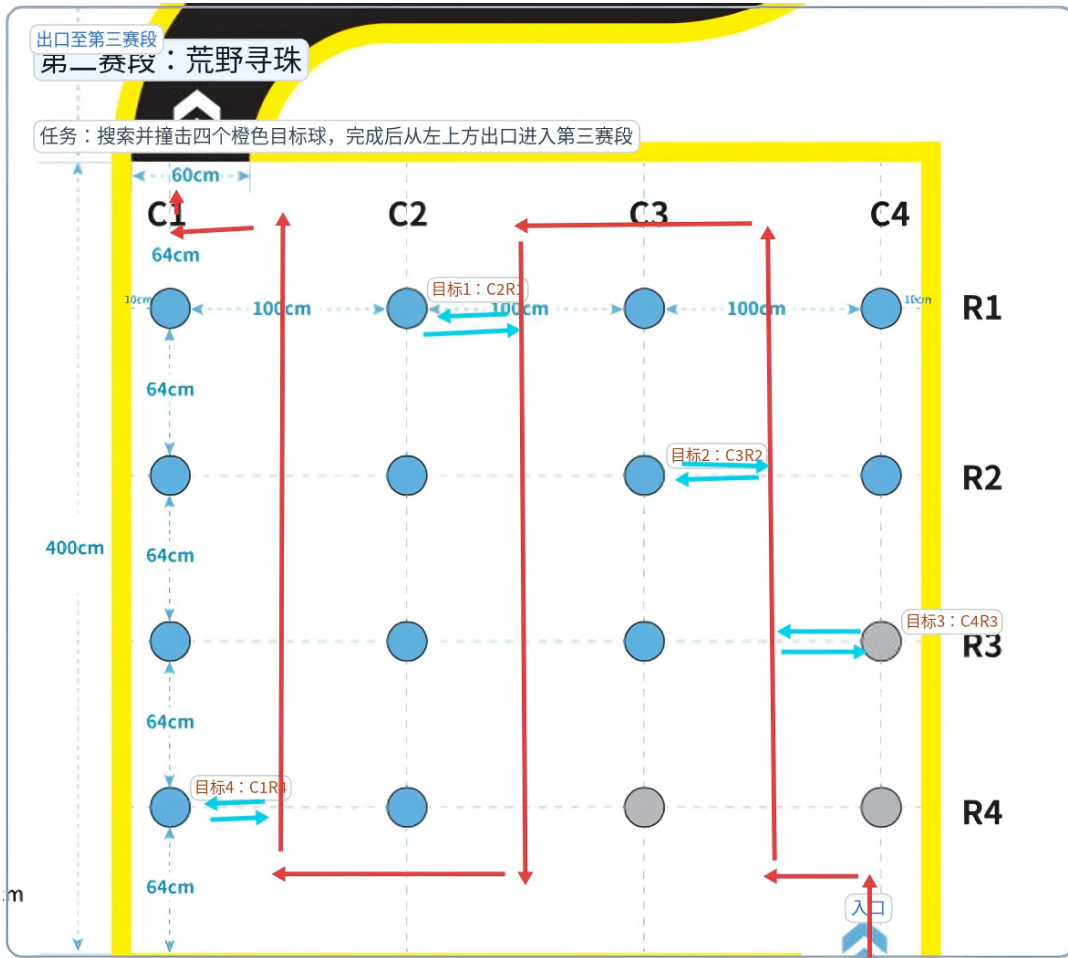


图 8: 第二赛段小球搜索路线

### 4.3 第三赛段：曲道冲锋

第三赛段的任务是沿曲线赛道行进到第四赛段入口。项目没有采用单纯的侧边界跟随，而是把曲道采样为一组世界坐标 waypoint。机器人从当前点找到合适的起始路点，然后逐点跟踪。

该策略的好处是路径清晰，调试直观。若某个弯道处容易偏离，只需要调整附近 waypoint 或速度限制，而不必重写整体控制逻辑。机器人到达每个路点后会根据目标航向进行必要转向，使进入下一段路时姿态更接近预期。

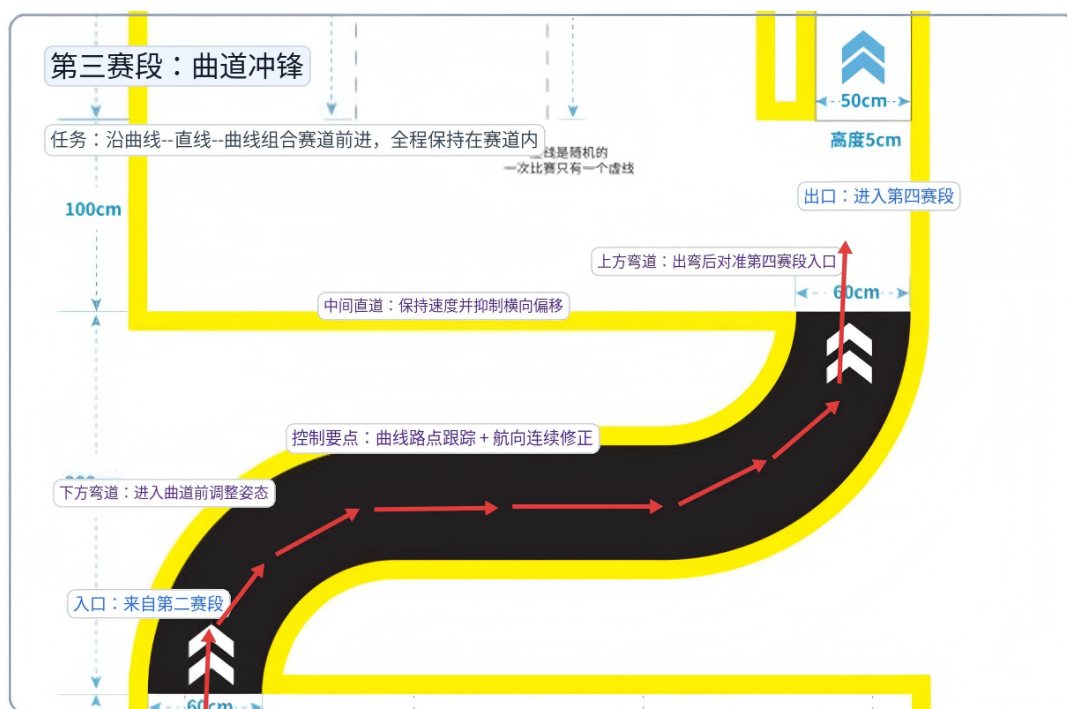


图 9: 第三赛段曲线路点

#### 4.4 第四赛段：深隧寻珍

第四赛段是任务最复杂的部分。机器人需要在通道中寻找可乐瓶、橙色小球和足球，完成语音播报和交互动作，同时识别限高杆和障碍物。

项目采用蛇形路径覆盖三条竖向通道。路径从入口进入左侧通道，再转移到中间通道和右侧通道，依次接近三个目标物体所在位置。每到达目标附近，系统进行语音播报，并执行对应动作：可乐瓶通过身体撞倒，橙色球通过接近撞击使其晃动，足球通过连续接触使其朝门框方向移动。

对于限高杆，项目使用低姿态通过策略。机器人在接近限高区域前降低身体高度，调整俯仰角，并执行低姿态步态序列。对于无法跨越障碍，系统在路径设计中预留绕行点，尽量避免正面硬碰。

这一段体现了项目中“先保证路线，再完成交互”的思路。目标识别和交互很重要，但机器人必须先保持可控，否则识别结果也无法转化为有效动作。



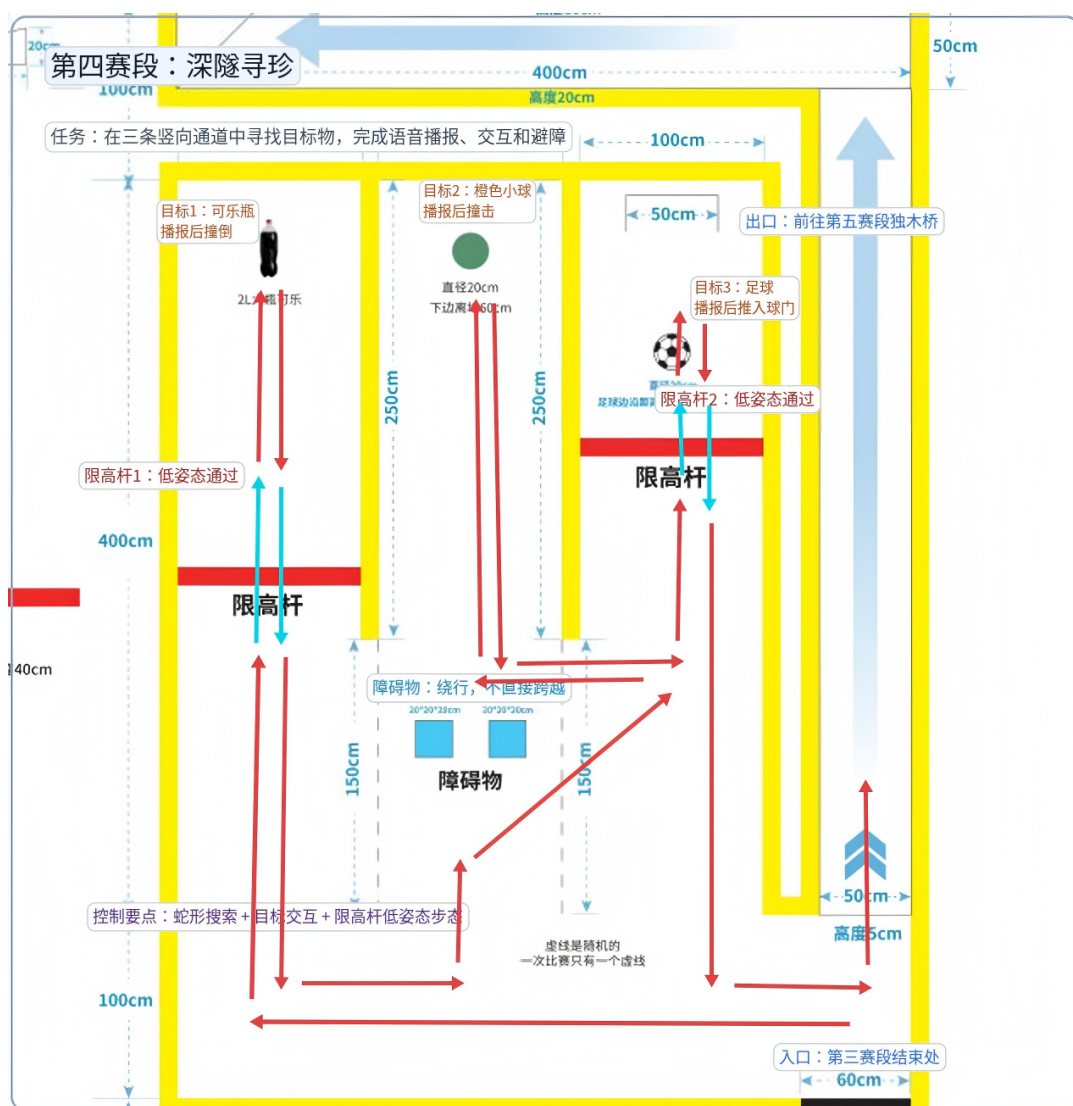


图 10: 第四赛段深隧搜索路径

#### 4.5 第五赛段：孤梁稳渡

第五赛段包含上坡、侧倾桥面、多次转向、平地段和跳下动作。项目在赛段文件中将桥面拆成五个 leg，每个 leg 对应不同的地形和控制参数。

上坡段中，机器人沿桥中心线前进，并根据进度调整身体俯仰和步高。侧倾段中，系统读取横滚角，叠加横滚补偿和横向速度修正，尽量保持机器人不向低侧滑落。转向时不简单追求快速原地旋转，而是采用较慢的稳定转向方式，以减少支撑不稳。

到达桥末端后，机器人执行跳下动作，并在落地后进行姿态修正。该段的设计重点是稳定性。相比平地赛段，桥面容错空间小，任何过大的速度和角速度都可能带来明显风险。





图 11: 第五赛段独木桥通过策略

#### 4.6 第六赛段：掘金建功

第六赛段要求将足球从出口位置带出或踢出，并到达终点趴下。项目采用固定起点校准策略，将第五赛段落地后的位姿校准为第六赛段的起始参考。随后，系统根据已知足球位置规划接近路径。

机器人先使用宽步态朝足球接近。宽步态可以减少腿部直接把球踢偏的概率，使机器人更容易把足球置于身体控制范围内。接近后，系统切换为窄步态，沿出口方向带球前进。最后，机器人向前释放足球，移动到终点区域，并执行趴下动作。

该策略把一个不稳定的接触任务变成了连续可调的运动过程。即使足球在带出过程中有小幅偏移, 机器人也可以通过航向修正和横向速度调整继续完成任务。

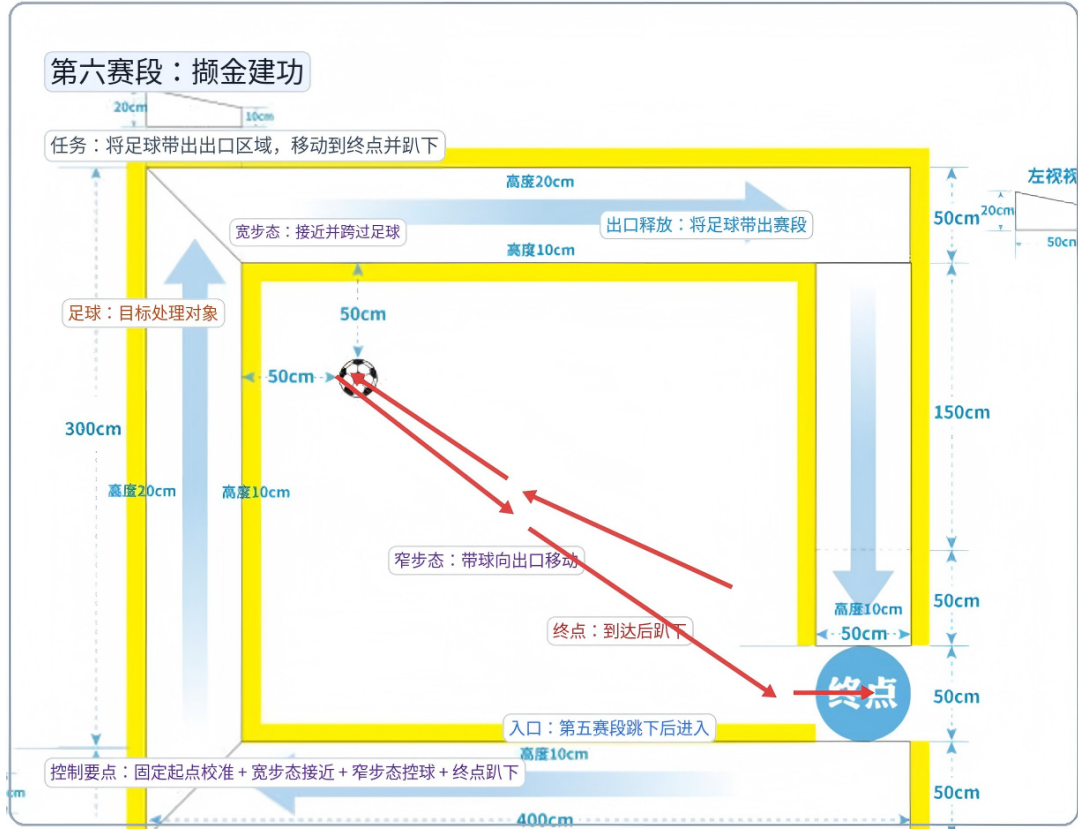


图 12: 第六赛段足球带出路线

## 5 系统实现

### 5.1 项目代码结构

项目主体位于 `project3130810-385179/cyberdog_race`。其中, `navigation/segments` 是过关逻辑最集中的目录, 六个赛段分别对应六个 Python 文件。这样的目录组织使调试流程比较清楚: 如果某一赛段失败, 可以直接进入对应文件查看路径、步态和参数。

表 4: 核心代码文件说明

| 文件                                       | 说明                        |
|------------------------------------------|---------------------------|
| <code>cyberdog_race/race_main.py</code>  | 比赛主入口, 负责启动控制、感知、语音和状态机。  |
| <code>navigation/state_machine.py</code> | 比赛状态机, 实现六个赛段的顺序调度。       |
| <code>navigation/course_map.py</code>    | 赛道坐标和路点配置。                |
| <code>navigation/navigator.py</code>     | 通用世界坐标导航器。                |
| <code>utils/nav_helper.py</code>         | 到点、转向、雷达横向修正和斜坡转向等导航辅助函数。 |

| 文件                            | 说明                   |
|-------------------------------|----------------------|
| lcm_controller.py             | LCM 控制器封装，提供运动命令接口。  |
| perception/ros2_perception.py | ROS2 感知节点，统一管理传感器数据。 |
| perception/ball_detector.py   | 小球检测模块。              |
| perception/object_detector.py | 物体检测模块。              |
| segments/segment_1.py         | 第一赛段石板路通过策略。         |
| segments/segment_2.py         | 第二赛段小球搜索与撞击策略。       |
| segments/segment_3.py         | 第三赛段曲道路点跟踪策略。        |
| segments/segment_4.py         | 第四赛段深隧寻物、低姿态钻杆和交互策略。 |
| segments/segment_5.py         | 第五赛段桥面、侧倾补偿和跳下策略。    |
| segments/segment_6.py         | 第六赛段宽窄步态控球和终点趴下策略。   |

5.2 主程序入口实现

race\_main.py 支持仿真模式、实机模式、单赛段测试和低姿态测试。完整比赛流程中，主程序首先启动 LCM 控制器，再创建语音模块和 ROS2 感知节点。感知节点启动后，主程序等待传感器就绪，并执行坐标校准。最后创建 RaceStateMachine 并调用 run()。

这种入口设计兼顾了正式比赛和调试需求。正式运行时可以直接跑全流程；调试时可以通过参数只测试某个赛段，不必每次从起点跑到目标段。

5.3 状态机实现

状态机的实现保持简洁。每个状态处理函数调用对应赛段的 execute()，返回完成后切换状态。状态机本身不直接处理石板、足球或限高杆细节，而是专注于比赛流程管理。

这种分工避免了主流程文件不断膨胀。赛段越复杂，越需要保持调度层稳定。只要每个赛段遵守输入和返回约定，状态机就可以持续复用。

5.4 分赛段模块实现

六个赛段文件都采用相似接口：

```
execute(ctrl, perception, ...)
```

其中 `ctrl` 是运动控制器, `perception` 是感知对象, 部分赛段还接收 `speaker` 和 `navigator`。各赛段内部根据任务需要定义本段参数, 例如速度、容差、目标坐标、步高、身体高度和超时时间。

这种写法的优点是参数集中。调试时, 常见修改集中在赛段文件开头的参数区, 不需要在多个模块之间来回查找。

## 5.5 配置文件与参数管理

项目使用 `config.toml` 和若干步态配置文件保存部分参数。第五、第六赛段中还会在运行时发布步态参数, 用于调整身体高度和腿部偏移。对于低姿态和桥面等特殊任务, 项目保留了独立的步态配置文件, 便于单独试验和回滚。

参数管理的原则是: 通用参数放在公共模块, 赛段特有参数放在赛段文件, 特殊步态参数放在配置文件或本段常量中。这样既方便快速调参, 也能避免不同赛段之间相互影响。

## 5.6 语音播报实现

第四赛段要求识别目标和障碍后语音播报。项目通过 `VoiceSpeaker` 封装语音输出, 并在赛段策略中根据目标类型调用播报接口。若语音模块初始化失败, 系统不会因此停止比赛, 而是打印日志并继续执行动作。

这一处理符合比赛系统的优先级: 语音是计分任务的一部分, 但机器人运动安全和流程推进仍然需要保持可控。

# 6 实验结果与分析

## 6.1 仿真测试结果

系统在仿真环境中完成了六个赛段的连续运行验证。测试过程中, 机器人能够按照状态机顺序完成站立初始化、各赛段导航与动作执行, 并在第六赛段结束后进入终点趴下状态。整体流程未出现需要人工接管的情况, 说明主控入口、感知读取、状态机调度、赛段策略和 LCM 运动控制之间的接口能够正常协同。

从测试结果看, 系统的主要优势在于流程清晰、异常边界明确。每个赛段都有独立的完成条件和超时保护, 即使某个局部动作存在误差, 也不会影响状态机的整体结构。对于视觉不稳定的场景, 系统保留了基于赛道几何的降级策略, 使比赛流程具备更好的连续性。

6.2 各赛段通过情况

表 5: 各赛段测试通过情况记录表

| 赛段   | 主要任务  | 通过情况 | 备注                                    |
|------|-------|------|---------------------------------------|
| 第一赛段 | 石板通行  | 正常通过 | 机器人使用高抬腿步态通过石板区域，航向修正和横向修正能够保持行进方向稳定。 |
| 第二赛段 | 橙色球撞击 | 正常通过 | 按预设搜索与撞击策略完成目标小球处理，并顺利进入第三赛段入口。       |
| 第三赛段 | 曲道行进  | 正常通过 | 机器人按照曲线路点连续行进，能够在弯道和直道之间完成姿态调整。       |
| 第四赛段 | 深隧寻物  | 正常通过 | 完成目标物体交互、语音播报和限高杆低姿态通过，通道内导航过程保持可控。   |
| 第五赛段 | 独木桥通过 | 正常通过 | 机器人完成上坡、侧倾桥面和平地段通过，并在桥末执行跳下动作。        |
| 第六赛段 | 足球带出  | 正常通过 | 机器人使用宽步态接近足球、窄步态带球出线，并在终点区域完成趴下动作。    |

6.3 全流程耗时分析

表 6: 全流程耗时统计表

| 阶段   | 耗时       | 说明                                           |
|------|----------|----------------------------------------------|
| 初始化  | 5s       | 包括参数解析、LCM 控制器启动、感知节点初始化、传感器就绪检查、坐标校准和机器人站立。 |
| 第一赛段 | 20s      | 从起点站立后通过石板区域，并完成进入第二赛段前的转向。                  |
| 第二赛段 | 1min 39s | 完成小球搜索、目标撞击和出口导航。                            |
| 第三赛段 | 41s      | 按曲线路点通过曲线-直线-曲线组合赛道。                         |
| 第四赛段 | 3min 20s | 完成深隧内目标交互、语音播报、障碍绕行和限高杆通过。                   |
| 第五赛段 | 2min 42s | 完成独木桥上坡、侧倾桥面、转向、平地段和跳下动作。                    |
| 第六赛段 | 58s      | 完成足球接近、带出、释放、终点移动和趴下动作。                      |
| 总计   | 9min 35s | 全流程耗时应控制在赛题规定的 15 分钟以内。                      |

耗时统计表中的具体数值可根据最终测试日志填写。由于比赛成绩同时受完成度、耗时和行进过程稳定性影响，记录单段耗时有助于定位后续优化重点。例如，若第二或第四赛段耗时偏长，应优先优化目标搜索和交互动作；若第五赛段耗时偏长，则应重点检查桥面速度、转向策略和横滚补偿参数。

6.4 稳定性分析

从系统设计看，稳定性主要来自三个方面。

第一，使用世界坐标路点减少了纯时间控制带来的累积误差。机器人不是盲目走固定秒数，而是根据当前位置持续修正。

第二，各赛段都设置了速度、容差和超时。速度限制降低了失稳风险，容差避免机器人在小范围误差内反复震荡，超时避免局部任务无限等待。

第三，特殊地形采用特殊步态。石板、限高杆、倾斜桥面和足球处理都没有强行使用同一种运动方式，而是根据任务重新调整步高、身体高度、姿态或腿部间距。

6.5 问题与改进

当前方案仍有不足。第二赛段的几何扫场依赖赛道先验，在目标位置随机性较强时需要进一步增强视觉识别稳定性。第四赛段目标交互仍然依赖较多预设坐标，如果目标位置变化较大，机器人需要更主动的视觉搜索和局部避障能力。第五赛段的侧倾补偿参数需要在实测中继续调整，过弱会滑移，过强可能导致姿态不自然。第六赛段控球策略相对稳妥，但耗时可能偏长，需要在速度和可靠性之间继续权衡。

这些问题不是独立存在的。它们反映了四足机器人比赛中的共同矛盾：感知越灵活，控制越难预测；动作越快，稳定性越难保证；策略越保守，比赛耗时越长。后续优化应围绕这一矛盾展开，而不是单纯追求某个模块的复杂度。

7 总结与展望

本项目围绕“荒野寻宝”赛题设计了一套 CyberDog 自主竞赛系统。系统以状态机组织比赛流程，以世界坐标路点完成主要导航，以 ROS2 感知节点统一传感器输入，以 LCM 控制器封装运动命令，并针对六个赛段分别设计了石板高抬腿、几何扫场、曲道路点跟踪、低姿态钻杆、桥面横滚补偿和宽窄步态控球等策略。

项目的主要价值在于把复杂任务拆成了可执行的工程闭环。它没有把问题简单理解为“识别目标”或“走到终点”，而是把机器人在赛道中的位置、姿态、地形、动作和任务完成条件放在一起考虑。这样的设计更贴近真实机器人系统的工作方式，也更能反映具身智能应用中“能稳定完成任务”的实际意义。

后续可以从三个方向继续改进。第一，提高视觉识别和目标定位的可靠性，使第二、第四赛段减少对固定坐标的依赖。第二，引入更细的局部避障和轨迹调整能力，提高狭窄通道和随机目标位置下的适应性。第三，继续优化特殊步态参数，尤其是低姿态通过和倾斜桥面行走，使系统在实机环境中具备更好的稳定性。

A 主要参数表

表 7: 部分关键参数与含义

| 参数类别  | 示例参数              | 含义                     |
|-------|-------------------|------------------------|
| 路点容差  | WAYPOINT_TOL_M    | 判断是否到达 waypoint 的距离阈值。 |
| 航向容差  | HEADING_TOL_DEG   | 判断是否朝向目标方向的角度阈值。       |
| 石板步高  | STONE_STEP_H_MAX  | 第一赛段通过石板时的足端抬高参数。      |
| 低姿态高度 | _LOW_CRAWL_HEIGHT | 第四赛段通过限高杆时的身体高度。       |

| 参数类别  | 示例参数        | 含义                     |
|-------|-------------|------------------------|
| 侧倾速度  | SPEED_TILT  | 第五赛段倾斜桥面行走速度。          |
| 横滚补偿  | ROLL_GAIN   | 根据 IMU 横滚角生成补偿姿态的比例系数。 |
| 宽步态编号 | GAIT_WIDE   | 第六赛段接近足球时使用的步态。        |
| 窄步态编号 | GAIT_NARROW | 第六赛段带球时使用的步态。          |

B 代码目录结构

```
cyberdog_race/  
  race_main.py  
  lcm_controller.py  
  config.toml  
  navigation/  
    state_machine.py  
    navigator.py  
    course_map.py  
    segments/  
      segment_1.py  
      segment_2.py  
      segment_3.py  
      segment_4.py  
      segment_5.py  
      segment_6.py  
  perception/  
    ros2_perception.py  
    detectors/  
      ball_detector.py  
      object_detector.py  
      boundary_detector.py  
      bridge_detector.py  
  utils/  
    nav_helper.py  
    speech.py
```



C 关键函数说明

表 8: 关键函数说明

| 函数                                             | 说明                  |
|------------------------------------------------|---------------------|
| <code>run_full_race()</code>                   | 启动完整比赛流程。           |
| <code>RaceStateMachine.run()</code>            | 状态机主循环，负责状态切换和超时判断。 |
| <code>go_to_waypoint()</code>                  | 根据世界坐标目标点生成运动命令。    |
| <code>turn_to_angle()</code>                   | 控制机器人转向指定航向角。       |
| <code>LCMController.locomotion()</code>        | 发送连续运动控制命令。         |
| <code>LCMController.set_body_pose()</code>     | 调整身体高度和俯仰角。         |
| <code>segment_5._run_lateral_tilt_leg()</code> | 第五赛段倾斜桥面行走控制。       |
| <code>segment_6._carry_ball_to_exit()</code>   | 第六赛段窄步态带球出线控制。      |

D 赛段 waypoint 配置表

表 9: 赛段入口坐标

| 赛段 | $x / \text{m}$ | $y / \text{m}$ | 航向角 / deg |
|----|----------------|----------------|-----------|
| 1  | 0.0            | 0.0            | 0.0       |
| 2  | 3.0            | 0.0            | 90.0      |
| 3  | -0.3           | 4.0            | 90.0      |
| 4  | 3.0            | 7.0            | 180.0     |
| 5  | 3.0            | 7.0            | 90.0      |
| 6  | 3.0            | 13.0           | -90.0     |